← Back to Client Libraries

MONGODB EXTENSION FOR HIBERNATE ORM

Current

Overview
Get Started
Model Data
Entities
Interact with Data
Feature Compatibility
Issues & Help
Version Compatibility
API Documentation

Create Entities to Represent Collections

Copy page

Rate this page

On this page

Overview
MongoDB BSON Fields
Define an Entity
Example
Embedded Data
One-to-One Relationships
One-to-Many Relationships
Additional Information

Overview

In this guide, you can learn how to create Hibernate ORM **entities** that represent MongoDB collections. Entities are Java classes that define the structure of your data. When using the Hibernate ORM extension, you can map each entity to a MongoDB collection and use these entities to interact with the collection's documents.

TIP

Entities Tutorial

To view a tutorial that shows how to model one-to-many relationships by using entities and the Hibernate ORM extension, see the Modeling Relationships With Hibernate ORM and MongoDB Foojay blog post.

MongoDB BSON Fields

MongoDB organizes and stores documents in a binary representation called BSON that allows for flexible data processing. This section describes the Hibernate ORM extension's support for BSON fields, which you can include in your entities.

TIP

To learn more about how MongoDB stores BSON data, see BSON Types in the MongoDB Server manual.

The following table describes supported BSON field types and their Hibernate ORM extension equivalents that you can use in your Hibernate ORM entities:

BSON Field Type	Extension Field Type	BSON Description
`null`	`null`	Represents a null value or absence of data.
`Binary`	`byte[]`	Stores binary data with subtype 0.
`String`	`char`, `java.lang.Character`, `java.lang.String`, or `char[]`	Stores UTF-8 encoded string values.
`Int32`	`int` or `java.lang.Integer`	Stores 32-bit signed integers.
`Int64`	`long` or `java.lang.Long`	Stores 64-bit signed integers.
`Double`	`double` or `java.lang.Double`	Stores floating-point values.
`Boolean`	`boolean` or `java.lang.Boolean`	Stores `true` or `false` values.
`Decimal128`	`java.math.BigDecimal`	Stores 28-bit decimal values.
`ObjectId`	`org.bson.types.ObjectId`	Stores unique 12-byte identifiers that MongoDB uses as primary keys.
`Date`	`java.time.Instant`	Stores dates and times as milliseconds since the Unix epoch.
`Object`	`@org.hibernate.annotations.Struct` aggregate embeddable	Stores embedded documents with field values mapped according to their respective types. `@Struct` aggregate embeddables might also contain array or `Collection` attributes.

`Array`	`Array`, `java.util.Collection` (or subtype) of supported types	Stores array values with elements mapped according to their respective types. Character arrays require setting the `hibernate.type.wrapper_array_handling` configuration property.

Define an Entity

To create an entity that represents a MongoDB collection, create a new Java file in your project's base package directory and add your entity class to the new file. In your entity class, specify the fields you want to store and the collection name. The `name` element of the `@jakarta.persistence.Table` annotation represents your MongoDB collection name. Use the following syntax to define an entity:

```
@Entity
@Table(name = "<collection name>")
public class <EntityName> {
    @Id
    // Specify your primary key field here
    private <field type> <field name>;

    // Include additional fields here
    private <field type> <field name>;

    // Parameterized constructor
    public <EntityName>(<parameters>) {
        // Initialize fields here
    }

    // Default constructor
    public <EntityName>() {

    }

    // Getter and setter methods
    public <field type> get<FieldName>() {
        return <field name>;
    }

    public void set<FieldName>(<field type> <field name>) {
        this.<field name> = <field name>;
    }
}
```

To use your entities, you can query them in your application files. To learn more about CRUD operations in the Hibernate ORM extension, see the Perform CRUD Operations guide.

Example

This sample `Movie.java` entity class defines a `Movie` entity that includes the following information:

- `@Entity` annotation that marks the class as a Hibernate ORM entity
- `@Table` annotation that maps the entity to the `movies` collection from the Atlas sample datasets
- `@Id` and `@ObjectIdGenerator` annotations that designate the `id` field as the primary key and configure automatic `ObjectId` generation

  > **TIP**
  >
  > **Primary Key Values**
  >
  > This example specifies the `ObjectId` field as the entity's primary key, but you can also set `String`, `Int`, or `UUID` fields as the primary key by using the `@Id` annotation.

- Private fields that represent movie data
- Default and parameterized constructors for entity instantiation
- Getter and setter methods that provide access to the entity's fields

```
package org.example;

import com.mongodb.hibernate.annotations.ObjectIdGenerator;
```

```
import org.bson.types.ObjectId;
import java.util.List;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "movies")
public class Movie {
        @Id
        @ObjectIdGenerator
        private ObjectId id;
        private String title;
        private String plot;
        private int year;
        private List<String> cast;


        public Movie(String title, String plot, int year, List<String> cast)
                this.title = title;
                this.plot = plot;
                this.year = year;
                this.cast = cast;
        }

        public Movie() {

        }

        public ObjectId getId() {
                return id;
        }

        public String getTitle() {
                return title;
        }

        public void setTitle(String title) {
                this.title = title;
        }

        public String getPlot() {
                return plot;
        }

        public void setPlot(String plot) {
                this.plot = plot;
        }

        public int getYear() {
                return year;
        }

        public void setYear(int year) {
                this.year = year;
        }

        public List<String> getCast() {
                return cast;
        }

        public void setCast(List<String> cast) {
                this.cast = cast;
        }
}
```

> **TIP**
>
> To learn more about the fields used in the entity class definition, see the MongoDB BSON Fields section of this guide.

Embedded Data

The Hibernate ORM extension supports embedded documents through Hibernate ORM `@Embeddable` annotations. With embedded documents, you can create One-to-Many, Many-to-One, and One-to-One relationships within MongoDB documents. This format is ideal for representing data that is frequently accessed together.

To represent embedded documents, use the `@Struct` and `@Embeddable` annotations on a class to create a `@Struct` aggregate embeddable. Then, include the embeddable type in your parent entity as a field. The Hibernate ORM extension supports embedding single objects, arrays, and collections of embeddables.

> **TIP**
>
> To learn more about `@Struct` aggregate embeddables, see @Struct aggregate embeddable mapping in the Hibernate ORM documentation.

One-to-One Relationships

A One-to-One relationship is when a record in one database is associated with exactly one record in another database. In MongoDB, you can create a collection with an embedded document field to model a One-to-One relationship. The Hibernate ORM extension allows you to create embedded document fields by using `@Struct` aggregate embeddables.

The example defines a field with a `@Struct` aggregate embeddable type in an entity similar to the Define an Entity example in this guide. The sample `Movie.java` entity class includes the following information:

- `@Entity` and `@Table` annotations that define the entity and map it to the `movies` collection
- `@Id` and `@ObjectIdGenerator` annotations that designate the `id` field as the primary key
- String field that represents the movie's title
- `@Struct` aggregate embeddable fields that represent movie awards and studio information

The following example represents a One-to-One relationship because each `Movie` entity is associated with one `Awards` embeddable and one `Studio` embeddable:

```
@Entity
@Table(name = "movies")
public class Movie {

    @Id
    @ObjectIdGenerator
    private ObjectId id;
    private String title;
    private Awards awards;
    private Studio studio;

    public Movie(String title, Awards awards, Studio studio) {
        this.title = title;
        this.awards = awards;
        this.studio = studio;
    }

    public Movie() {

    }

    // Getter and setter methods

}
```

The following sample code creates an `Awards` `@Struct` aggregate embeddable:

```
@Embeddable
@Struct(name = "Awards")
public class Awards {
    private int wins;
    private int nominations;
    private String text;

    public Awards(int wins, int nominations, String text) {
       this.wins = wins;
       this.nominations = nominations;
       this.text = text;
```

```
    }

    public Awards() {

    }

    // Getter and setter methods

}
```

The following sample code creates a `Studio` `@Struct` aggregate embeddable:

```
@Embeddable
@Struct(name = "Studio")
public class Studio {
    private String name;
    private String location;
    private int foundedYear;

    public Studio(String name, String location, int foundedYear) {
        this.name = name;
        this.location = location;
        this.foundedYear = foundedYear;
    }

    public Studio() {

    }

    // Getter and setter methods

}
```

One-to-Many Relationships

A One-to-Many relationship is when a record in one database is associated with many records in another database. In MongoDB, you can define a collection field that stores a list of embedded documents to model a One-to-Many relationship. The Hibernate ORM extension allows you to create embedded document fields by using a list of `@Struct` aggregate embeddables.

The example defines a field that stores a list of `@Struct` aggregate embeddables in an entity similar to the Define an Entity Example in this guide. The sample `Movie.java` entity class includes the following information:

- `@Entity` and `@Table` annotations that define the entity and map it to the `movies` collection
- `@Id` and `@ObjectIdGenerator` annotations that designate the `id` field as the primary key
- String field that represents the movie's title
- List field that stores multiple `Writer` `@Struct` aggregate embeddables, which represents writer information

The following example represents a One-to-Many relationship because each `Movie` entity is associated with multiple `Writer` embeddables:

```
@Entity
@Table(name = "movies")
public class Movie {
    @Id
    @ObjectIdGenerator
    @Column(name = "_id")
    private ObjectId id;
    private String title;
    private List<Writer> writers;

    public Movie(String title, List<Writer> writers) {
        this.title = title;
        this.writers = writers;
    }

    public Movie() {

    }
```

```
    // Getter and setter methods

}
```

The following sample code creates a `Writer` `@Struct` aggregate embeddable:

```
@Embeddable
@Struct(name = "Writer")
public class Writer {
    private String name;

    public Writer() {
    }

    public Writer(String name) {
        this.name = name;
    }

    // Getter and setter methods

}
```

Additional Information

To learn how to use your entities to run database operations, see the following guides in the Interact with Data section:

- Perform CRUD Operations
- Specify a Query

To learn more about Hibernate ORM fields, see the Mapping types section in the Hibernate ORM documentation.

To learn more about Hibernate ORM entities, see POJO Models in the Hibernate ORM documentation.

Back
Get Started

Next
CRUD Operations